TEST REPORT

MyPowerPlant Testing & Verification

Automated test suite construction, execution results, and defects identified across the React frontend and FastAPI backend.

244	244	3	177
TESTS RUN	PASSING	DEFECTS FOUND	TESTS ADDED

PROJECT	BRANCH	DATE
MyPowerPlant	**sec**	**10 September 2026**
SCOPE	STACKS	STATUS
Automated unit & integration	**React 19 / TS · FastAPI**	**Suite green, 3 defects open**

— Contents

1	Executive summary	What was built, what it found
2	Testing approach	Levels, techniques, risk prioritisation
3	Automated test results	244 tests across two stacks
4	Defects identified	Two Major, one Moderate
5	Verification by hand calculation	Independent check of the KPI engine
6	Continuous integration	How the evidence stays in the build
7	Testing not yet executed	Manual and usability status
8	Coverage gaps and recommendations	What to do next
9	Appendix A — Test inventory	Full suite breakdown
10	Appendix B — Reproducing these results	Commands

1 Executive summary

This report covers the construction and execution of an automated test suite for MyPowerPlant, a Victorian household solar, battery and VPP profiling application. Before this work the backend held 67 tests and the frontend had **no test runner at all** — the validation rules and KPI calculations that drive the product's financial estimates had no automated regression protection.

177 frontend tests were added across two suites, bringing the project to **244 automated tests, all passing**. Executing them surfaced **three defects**, two of which materially affect the numbers shown to users.

SUITE	LAYER	TESTS	RESULT
`validateProfile.test.ts`	Frontend · new	83	**Pass**
`kpis.test.ts`	Frontend · new	94	**Pass**
`test_access_control.py`	Backend · existing	21	**Pass**
`test_input_validation.py`	Backend · existing	32	**Pass**
`test_headers_and_limits.py`	Backend · existing	14	**Pass**
Total		**244**	**All passing**

Backend per-file counts are apportioned from a 67-test collection across three files.

Key findings

The appliance energy model overstates refrigerator consumption by roughly 30×. A single fridge is modelled at 322 kWh per week against a real-world 7–14 kWh. For any household that does not enter a monthly kWh figure — the least informed user, and the one most reliant on the estimate — the headline consumption figure is badly wrong.

Households with no solar are shown dollar savings they are not making. A household that answers "no solar" still receives a $33.69 per week saving and 51.7 kg of avoided carbon, calculated from a hypothetical array. The engine flags this figure as potential; the dashboard does not read that flag before rendering it as a dollar amount.

Neither defect was detectable by inspection or by using the app — both required calculating expected values independently and comparing. That is the argument for the suite existing.

> **SCOPE STATEMENT**
>
> This report covers **automated testing only**. The manual functional cases (FUN-01…FUN-12) and the usability sessions are documented and ready but **have not been executed**. Section 7 states this position explicitly rather than implying broader coverage than was achieved.

2 Testing approach

MyPowerPlant has **no user accounts**. A household is addressed by a server-generated identifier held in the browser. That single design decision drives the prioritisation: the identifier *is* the access control, so its unguessability carries more risk than any feature.

2.1 Test levels

LEVEL	COVERS	LOCATION	CADENCE
Unit	Validation rules, KPI mathematics, formatters	`frontend/src/lib/`	Every push
Integration	API routes against a live DB session	`backend/tests/`	Every push
Security	Access control, input rejection, headers, rate limits	`backend/tests/`	Every push
System	Full browser journeys	Manual — FUN-01…FUN-12	Per iteration
Usability	Participants completing task scenarios unaided	Facilitated sessions	Per iteration

2.2 Design techniques

Test cases were derived from named techniques rather than by walking the interface and guessing. Each case records the technique that produced it, so the reasoning is auditable.

TECHNIQUE	APPLIED TO	EXAMPLE
Equivalence partitioning	Postcodes, allow-list fields	Victorian / non-Victorian / non-numeric — three classes, three cases, not thirty arbitrary postcodes
Boundary value analysis	All six bounded numeric fields	Panel count bounded 1–80, so 0, 1, 80, 81 are each tested
Negative testing	Server-side schema	Client-chosen identifiers, control characters, oversized payloads submitted directly to the API
Path coverage	Conditional fields	Dryer frequency appears only when `dryer = yes`; both branches tested
Independent recalculation	KPI engine	Expected values computed by hand from the methodology, then compared (Section 5)

2.3 Risk-based prioritisation

Effort was allocated by likelihood × impact, not spread evenly across features.

RISK	IMPACT	PRIORITY	MITIGATING TESTS
Data reachable by altering an identifier	Severe — privacy breach	P1	SEC-01…SEC-05
Identifiers guessable or sequential	Severe	P1	`test_access_control.py`

RISK	IMPACT	PRIORITY	MITIGATING TESTS
Invalid input accepted and corrupting KPIs	High	P1	VAL-01…VAL-28
User abandons the five-step wizard	High — product unusable	P1	Usability US-1, US-2
KPI figures wrong or misleading	High — drives a purchase decision	P2	FUN-07…FUN-10
Missing headers or rate limits	Medium	P2	SEC-11, SEC-12

WHY THE KPI ENGINE WAS TESTED THIS HARD

The KPIs inform a real financial decision — whether to spend thousands on solar or a battery. "It rendered a number" is not evidence the number is right. Both Major defects in Section 4 sit in this area and neither is visible from the interface.

3 Automated test results

Both suites were executed on 10 September 2026 against branch `sec`. Raw output is retained in `testing/05-evidence/`.

3.1 Frontend — Vitest

```
$ npm test

 ✓ src/lib/validateProfile.test.ts  (83 tests)   9ms
 ✓ src/lib/kpis.test.ts             (94 tests)  20ms

 Test Files  2 passed (2)
      Tests  177 passed (177)
   Duration  403ms
```

3.2 Backend — pytest

```
$ pytest tests/ -v

 ======================== 67 passed, 3 warnings in 0.43s ========================
```

3.3 Coverage by area

AREA	CASE IDS	TESTS	NOTES
Postcode equivalence classes	VAL-01…06	14	Includes all eight non-Victorian state prefixes
Numeric boundary analysis	VAL-07…14	40	Six fields × min−1 / min / max / max+1 / non-numeric / empty
Allow-list fields	VAL-15…20	9	Battery models and energy providers
Conditional field paths	VAL-21…24	5	Dryer and solar branches, both directions
Required-field gating	FUN-03	14	Every required field blocks its step
KPI consumption & generation	FUN-08	10	Checked against hand calculations
Energy balance & battery shifting	FUN-09	7	Conservation of energy across the split
Retail rate & plan comparison	FUN-08	10	Includes division-by-zero guard
Appliance modelling	—	11	E = N × P × H and every conditional appliance
No-solar households	FUN-07	5	Surfaced defect KPI-02
Extreme-but-valid profiles	FUN-10	16	Four profiles × finiteness, plans, appliances, assumptions
Formatters	—	18	Boundary behaviour at 10 and 100
Access control	SEC-01…05	21	Backend — identifier issuance and retrieval

AREA	CASE IDS	TESTS	NOTES
Server-side input rejection	SEC-06…09	32	Backend — includes SQL-injection-shaped payloads
Headers, CORS, rate limits	SEC-11…13	14	Backend

3.4 Handling of known defects

Two tests are marked `it.fails`. They assert the behaviour believed to be *correct* for defects KPI-01 and KPI-02, and currently fail — so Vitest records them as passing while the defect stands. Each is paired with a test pinning the behaviour the code exhibits today.

When a defect is fixed, its `it.fails` case begins failing loudly, prompting removal of the marker and its paired pin. This keeps the suite green **without encoding the buggy behaviour as if it were correct** — a distinction that matters, because a test asserting a bug silently blesses it.

4 Defects identified

ID	SUMMARY	SEVERITY	STATUS
KPI-01	Refrigerator power rating overstates modelled consumption ~30×	S2 Major	Open
KPI-02	Households with no solar shown dollar savings and avoided carbon	S2 Major	Open
VAL-14	Battery unit minimum differs between client and server	S3 Moderate	Open

KPI-01 · Refrigerator power rating

S2 — Major

LOCATION `frontend/src/data/appliances.ts` — `refrigerator.powerKw`

FOUND BY `kpis.test.ts` → *a single refrigerator should use a realistic 5–20 kWh per week*

DETECTED VIA Independent plausibility check of modelled output

The refrigerator is assigned `powerKw: 1.92` — the identical value to the air conditioner — and run for 168 h/week. Running continuously is correct for a fridge; the power figure is not.

```
1.92 kW × 168 h = 322.56 kWh per refrigerator per week
```

A domestic refrigerator uses roughly 1–2 kWh *per day*, i.e. 7–14 kWh per week. The model is out by a factor of about 30. On the baseline two-fridge profile, refrigeration alone accounts for 645 kWh of a 687 kWh modelled week — **94% of total household consumption**, which is implausible on its face.

Cause. `1.92` appears to be a rated input power rather than an average draw. A fridge cycles at roughly a 15–25% duty cycle, giving an effective figure near 0.05 kW.

Impact. Affects only the modelled consumption path, used when a household has not entered a monthly kWh figure. That is precisely the least-informed user and the one most dependent on the estimate. Every downstream KPI inherits the error.

Recommended fix. Replace `powerKw` with a duty-cycle adjusted average draw, or retain rated power and set `typicalHoursPerWeek` to compressor run-hours. The first is clearer. Verify against the GEMS dataset the file cites.

KPI-02 · Savings shown to households without solar

S2 — Major

LOCATION `frontend/src/lib/kpis.ts` and `DashboardView.tsx:62`

FOUND BY `kpis.test.ts` → *a household with no solar should not be shown dollar savings*

DETECTED VIA FUN-07 — deliberate test of the no-system household

When a household answers `hasSolar: 'no'`, `buildKpiReport` still models a typical 6.6 kW array and carries those figures into `weekly.savingsAud` and `weekly.carbonAvoidedKg`. Measured output for a no-solar profile:

FIGURE	VALUE RETURNED
`generationKwh`	131.95 kWh
`savingsAud`	$33.69
`carbonAvoidedKg`	51.70 kg

The report does set `solarIsPotential: true`, but `DashboardView.tsx:62` renders `weekly.savingsAud` as a dollar KPI without consulting that flag. A household that has just told the product it has no panels is shown **$33.69 per week it is not saving** and 51.7 kg of carbon it is not avoiding.

Impact. The product's stated audience is households *considering* solar, so this is not an edge case — it is the main path for a large share of users. Presenting a hypothetical as an achieved figure could reasonably be read as misleading, which matters more than the arithmetic.

Recommended fix. Either zero these figures when `solarIsPotential` is true, or — better, since the potential figure is genuinely useful — relabel the cards in that case: "*You could save ~$34/week*" rather than "*You saved $34*". This is a product decision; the test currently pins existing behaviour so any change is deliberate.

VAL-14 · Client and server disagree on minimum battery units

S3 — Moderate

LOCATION `validateProfile.ts` vs `backend/app/models.py:165`

FOUND BY Cross-referencing both validation suites

The browser rejects `batteryUnits < 1`; the API schema permits `ge=0`. A value of `0` is therefore rejected in the interface and accepted by the API. The backend suite tests `-1` and `9` but has no case for `0`.

Impact. Low in practice, since the UI is the only caller. It matters as a policy question: should a household with no battery record zero units, or is "no battery" expressed another way? Whichever is right, the layers should agree.

Recommended fix. Decide the intended minimum, align both layers, and add a backend case for `0` asserting the decision.

5 Verification by hand calculation

To verify the KPI engine independently rather than merely observing that it produces output, expected values for a representative household were calculated by hand from the documented methodology, then compared against the engine.

INPUTS — BASELINE KEW HOUSEHOLD

FIELD	VALUE	FIELD	VALUE
Postcode	3101	Solar	12 panels × 400 W, facing S
Monthly bill	$180	Battery	Tesla Powerwall 2, 20% reserve
Monthly usage	540 kWh	Provider	AGL Energy
Peak period	Evening	Grid factor	0.77 kg CO_2-e/kWh

HAND CALCULATION

```
1. Weekly consumption      540 ÷ 4                          = 135.0000 kWh

2. Solar generation        array  = 12 × 400 ÷ 1000       =   4.8 kW
                           daily  = 4.8 × 4.2 × 0.8 × 0.55 =   8.8704 kWh
                           weekly = 8.8704 × 7             =  62.0928 kWh

3. Self-consumption        direct   = min(62.0928, 135 × 0.4)  = 54.0000 kWh
                           leftover = 62.0928 − 54             =  8.0928 kWh
                           battery  = 1 × 13.5 × (1 − 0.20)    = 10.8000 kWh usable
                           shifted  = min(10.8, 4.0464, 54)    =  4.0464 kWh
                           self     = 54 + 4.0464              = 58.0464 kWh

4. Distributed to grid     62.0928 − 58.0464                  =  4.0464 kWh

5. Retail rate             180 ÷ 540                          = $0.333333 / kWh

6. Carbon avoided          4.0464 × 0.77                      =  3.1157 kg

7. Weekly savings          58.0464 × 0.333333 + 4.0464 × 0.18  = $20.0772
```

COMPARISON AGAINST ENGINE OUTPUT

KPI	HAND-WORKED	ENGINE	MATCH
Weekly consumption	135.0000 kWh	135.0000 kWh	✓
Weekly generation	62.0928 kWh	62.0928 kWh	✓
Self-consumed	58.0464 kWh	58.0464 kWh	✓
Distributed	4.0464 kWh	4.0464 kWh	✓
Retail rate	$0.333333	$0.333333	✓

KPI	HAND-WORKED	ENGINE	MATCH
Carbon avoided	3.1157 kg	3.1157 kg	✓
Weekly savings	$20.0772	$20.0772	✓

Result: the solar, battery-shifting, export and carbon paths agree with the documented methodology to six decimal places.

> **CAVEAT**
>
> This profile supplies a monthly kWh figure, so the appliance model is not used for the consumption headline. Defect KPI-01 affects only the modelled path — a household that leaves monthly kWh blank — and is therefore **not exercised by this example**. A second worked example covering that path would strengthen the evidence.

6 Continuous integration

Tests that are not run are worse than no tests, because they present the appearance of coverage. The suite is therefore wired into the existing `security` workflow rather than left as a local command.

JOB	RUNS	FAILS THE BUILD ON
`secret-scan`	gitleaks over full history	Any secret, including one removed in a later commit
`frontend-audit`	`npm audit` , `tsc -b` , `lint` , **`npm test`**	High-severity advisory, type error, lint error, failing unit test
`backend-audit`	`pip-audit --strict`	Any known vulnerable dependency
`security-tests`	`pytest tests/ -v`	Any failing test

The `npm test` step was added as part of this work. The workflow triggers on every push to `main` , `development` and `sec` , on every pull request, and weekly on a schedule so that a newly disclosed advisory in an unchanged dependency is still caught.

> **PLACEMENT DECISION**
>
> Executable tests deliberately live beside the code they test — `backend/tests/` and `frontend/src/lib/` — not inside the `testing/` documentation folder. `pytest.ini` sets `testpaths = tests` and CI runs `pytest tests/` ; a suite relocated under `testing/` would be collected by neither and would silently stop running.

7 Testing not yet executed

Stated plainly so that coverage is not overclaimed.

ACTIVITY	ARTEFACTS	STATUS
Manual functional cases FUN-01…FUN-12	Written with steps, expected results, priorities	**Not executed**
Usability sessions US-1…US-5	Goal-based scenarios, facilitator script, note template	**No sessions run**
Usability findings → user story revisions	Template with before/after story columns	**Awaiting sessions**
End-to-end browser automation	None	**Not built**

Execution columns, session notes and usability evidence were left **deliberately blank**. Recording a result that was not observed would make the entire body of evidence unreliable, including the parts that are genuine.

The usability material is complete and ready to run: five scenarios written as situations and goals rather than instructions (a scenario naming a button tests reading comprehension, not the design), a facilitator script with a redirect table for handling participant questions without coaching, and a note-taking template that separates observation from interpretation.

8 Coverage gaps and recommendations

GAP	RISK	RECOMMENDATION	PRIORITY
KPI-01 and KPI-02 open	Users shown materially wrong figures	Fix KPI-01 (data correction); decide KPI-02 policy (zero vs relabel)	P1
No end-to-end browser test	The primary journey has no automated guard	Playwright covering FUN-01 alone	P2
Usability sessions not run	Wizard abandonment risk unmeasured	Run US-1…US-5 with 5 participants	P1
CI uses SQLite, not Postgres	JSON column semantics and constraint text unverified	Verify locally before merge; consider a Postgres service container	P3
CI installs test deps inline	New test dependency must be added twice or CI breaks	Use `requirements-dev.txt` in the workflow	P3
Second worked example needed	Modelled-consumption path unverified by hand	Work a no-bill household example after KPI-01 is fixed	P2
`kpis.selftest.ts` superseded	Dead code that CI never runs	Delete once nothing imports it	P3

Closing assessment

The automated layer is now in reasonable shape: 244 tests covering every documented validation boundary, the KPI methodology, and the access-control model, all enforced on every push. The two Major defects it surfaced were invisible from the interface and would not have been found by manual testing, which is the clearest available evidence that the investment was worthwhile.

The material gap is human testing. The scenarios and script are ready; what remains is to run them and feed the findings back into the backlog.

A Appendix — Test inventory

Frontend · `validateProfile.test.ts` — 83 tests

GROUP	COVERS
baseline profile	All five wizard steps validate cleanly for a known-good profile
VAL-01…06 postcode	Victorian 3xxx/8xxx, interstate, malformed, empty, whitespace
Boundary analysis (6 fields)	monthlyBill, monthlyKwh, panelCount, panelWatts, reservePercent, batteryAge
VAL-14 battery units	1, 8, 9 and the 0 divergence
VAL-15…17 battery models	Allow-list, "Not sure" escape hatch, off-list guidance text
VAL-18…20 providers	Allow-list, "Other", "Not sure", off-list
VAL-21…24 conditionals	Dryer and solar branches in both directions
FUN-03 gating	11 required fields; all errors reported at once; no empty messages

Frontend · `kpis.test.ts` — 94 tests

GROUP	COVERS
parseCount	"6+" forms, junk input, never returns NaN
Timeframe scaling	daily = weekly ÷ 7, monthly = weekly × 4, currency parity
FUN-08 consumption	Billed path, modelled fallback, provenance labelling, recalculation
FUN-08 generation	Full solar formula, orientation ordering, unknown-direction fallback
Energy balance	Conservation, no over-distribution, carbon derivation, period split
Retail rate	Bill ÷ kWh, default fallback, division-by-zero guard
Plan comparison	Ordering, single recommendation, battery compatibility, savings formula
Appliance modelling	E = N × P × H, every conditional appliance, provenance flags
FUN-07 no solar	Finiteness, no battery shifting, potential flag, *KPI-02*
FUN-10 extremes	4 profiles × finiteness, non-negativity, plans, assumptions
Formatters	Boundary behaviour at 10 and 100 for kWh, AUD, kg
Report shape	Purity, no mutation of the input profile

Backend · pytest — 67 tests

FILE	CLASSES
`test_access_control.py`	TestIdentifierIssuance · TestRetrieval · TestWrites
`test_input_validation.py`	TestPostcode · TestReferenceDataAllowlists · TestNumericBounds · TestHostileInput · TestConditionalRequirements
`test_headers_and_limits.py`	TestSecurityHeaders · TestCors · TestRateLimiting · TestErrorDisclosure

B Appendix — Reproducing these results

FRONTEND

```
cd frontend
npm ci
npm test           # 177 tests
npm run test:watch # during development
npx tsc -b && npm run lint
```

BACKEND

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
pytest tests/ -v   # 67 tests
```

SUPPORTING DOCUMENTATION

DOCUMENT	CONTENTS
`testing/01-test-plan.md`	Scope, levels, techniques, risk table, entry/exit criteria
`testing/02-test-cases/`	Validation, functional and security cases; traceability matrix; defect log
`testing/03-usability/`	Scenarios, facilitator script, session template, findings log
`testing/04-automated/`	Suite locations, CI enforcement, known gaps
`testing/05-evidence/`	Raw run output and the worked KPI example